

Strings appear in almost every computer program, no matter the language. This data type is common, so it's no surprise there are many approaches to manipulating and formatting strings. This section highlights a couple of best practices.

Use Raw Strings If Your String Has Many Backslashes

Escape characters allow you to insert text into string literals that would otherwise be impossible to include. For example, you need the `\` in `'Zophie\'s chair'` so Python interprets the second quote as part of the string, not the symbol marking the end of the string. Because the backslash has this special escape meaning, if you want to put an actual backslash character in your string, you must enter it as `\\`.

Raw strings are string literals that have an `r` prefix, and they don't treat the backslash characters as escape characters. Instead, they just put the backslashes into the string. For example, this string of a Windows file path requires several escaped backslashes, which isn't very pythonic:

```
>>> # Unpythonic Example
>>> print('The file is in C:\\Users\\Al\\Desktop\\Info\\Archive\\Spam')
The file is in C:\\Users\\Al\\Desktop\\Info\\Archive\\Spam
```

This raw string (notice the `r` prefix) produces the same string value while being more readable:

```
>>> # Pythonic Example
>>> print(r'The file is in C:\\Users\\Al\\Desktop\\Info\\Archive\\Spam')
The file is in C:\\Users\\Al\\Desktop\\Info\\Archive\\Spam
```

Raw strings aren't a different kind of string data type; they're just a convenient way to type string literals that contain several backslash characters. We often use raw strings to type the strings for regular expressions or Windows file paths, which often have several backslash characters in them that would be a pain to escape individually with `\\`.

Format Strings with F-Strings